Contents

1 Claude Code Evals: Three-Part Blog Series Plan

1.1 1. What We Want To Produce

This document defines the plan for a **three-part blog series on Claude Code evals**.

The goal is to help technical readers move from a vague feeling that "we should test our agents" to a practical understanding of **what to evaluate, why it matters, and how to start**.

The series should be written for readers who are comfortable with AI, data science, software engineering, and technical workflows, but who may not have built evals for agents before.

The tone should be:

- **Pedagogical**: explain concepts clearly, without assuming expert knowledge of evals.
- **Practical**: focus on workflows people can actually build.
- **Reflective**: acknowledge the messy reality of agentic tools.
- **Grounded**: connect claims to the research report and specific references.
- **Sceptical without being cynical**: Claude Code is useful, but one good run is not evidence.

The series should avoid becoming a generic introduction to evals. It should be specifically about **Claude Code as an agentic coding and workflow environment**.

Claude Code is not just a chat interface. Anthropic describes Claude Code as an agentic coding tool that reads a codebase, edits files, runs commands, and integrates with development tools. It can help build features, fix bugs, automate development tasks, stage changes, create commits, use MCP tools, follow `CLAUDE.md`, use skills, use hooks, and run agents or subagents.

The central thesis of the series is:

> The goal of evals is not to evaluate everything. The goal is to stop being surprised by the same failure twice.

The series should teach that Claude Code evals are not only about **answer quality**. They are about whether the **whole workflow** behaved correctly.

That includes:

- the task Claude was given,
- the repository state it started from,
- the instructions it loaded,
- the tools it used,
- the commands it ran,
- the files it changed,
- the skills it invoked,
- the agents it delegated to,
- the hooks that did or did not fire,
- the final output it produced,
- and the final repository state it left behind.

1.2 2. Executive Summary Of The Three Posts

Post	Working title	Main question	Reader movement	Core takeaway
Part 1	**Why "It Worked Once" Is Not Evidence**	Why do Claude Code workflows need evals?	From anecdotal trust to workflow-level thinking	Claude Code is not just answering. It is acting inside an environment.
Part 2	**What You Actually Need To Test**	What surfaces of a Claude Code workflow can be evaluated?	From vague "evals" to a concrete evaluation map	The right eval depends on which part of the workflow you are trying to trust.
Part 3	**Building A Tiny Eval Suite That Actually Helps**	How do we build a first useful eval suite?	From concept to implementation	Start with one workflow, real tasks, clear failures, and small regression cases.

The three posts should feel like one continuous learning journey:

```
Part 1: The problem
  -> Claude Code is impressive, but one good run is not reliability.

Part 2: The map
  -> Claude Code has many evaluation surfaces: output, files, tools, skills, agents, hooks, and sta

Part 3: The method
  -> Start small. Pick one workflow. Collect examples. Define failure. Build evals around what actu
```

A useful short version of the arc:

```
Part 1 gives the mental model.
Part 2 gives the evaluation map.
Part 3 gives the practical starter kit.
```

1.3 3. Pedagogical Arc

The series should not start with YAML files, harnesses, LLM-as-judge prompts, or eval infrastructure. That would be too fast.

Many readers are likely to have used Claude Code or a similar coding agent informally. They may have had moments where it felt magical: it fixed a bug, wrote a script, explained a codebase, or produced a convincing summary.

They may also have seen the opposite: the agent edited the wrong files, skipped tests, misunderstood conventions, invented details, or claimed to be finished when the work was not actually done.

That lived experience is the right starting point.

The first post should begin with the **feeling of false confidence**:

> Claude Code worked once. It looked impressive. But can we trust it to work again?

From there, the series can gradually build the reader's understanding.

1.3.1 3.1 Why Part 1 Comes First

Part 1 should establish the core mental model.

The reader needs to understand that Claude Code is not just a model producing text. It is a workflowing system that can operate inside a repository.

That means the object being evaluated is not simply:

```
model + prompt
```

It is closer to:

```
model
+ user task
+ repository state
+ CLAUDE.md
+ memory
+ tools
+ permissions
+ MCP servers
+ skills
+ hooks
+ subagents
+ execution environment
+ transcript
+ final output
+ final repository state
```

This is the first conceptual shift.

If readers do not understand this, they will treat evals as answer grading. They will ask, "Was the final response good?" but miss the deeper question: "Did the workflow behave correctly?"

1.3.2 3.2 Why Part 2 Comes Second

Once the reader understands that Claude Code is a workflowing system, the natural question becomes:

> What parts of the workflow should we evaluate?

Part 2 should answer that question.

It should introduce the idea of **evaluation surfaces**.

Claude Code can fail at different layers:

- the final answer can be wrong,
- the code can fail tests,
- the repository can be left messy,
- the wrong files can be edited,
- tools can be used poorly,

- commands can fail silently,
- a skill can fail to trigger,
- a skill can trigger too often,
- a subagent can be given the wrong task,
- a hook can fail to block something unsafe,
- instructions can be ignored,
- costs and latency can become unreasonable.

Those are not one failure type. They require different evals.

Part 2 should therefore give the map:

```
If you care about correctness, evaluate the final output.
If you care about repository hygiene, evaluate the git diff and final state.
If you care about safe execution, evaluate commands and tool use.
If you care about repeatable workflows, evaluate skills.
If you care about delegation, evaluate subagents.
If you care about workflow controls, evaluate hooks and permissions.
```

1.3.3 3.3 Why Part 3 Comes Third

Only after the reader has the mental model and the map should the series move into implementation.

Part 3 should answer:

> How do I build my first useful Claude Code eval suite without turning it into enterprise theatre?

The post should be highly practical.

The recommendation should be:

```
Do not start by evaluating everything.
Start with one workflow that matters.
Collect 5-10 real examples.
Define success and failure.
Automate deterministic checks where possible.
Use judgement checks where necessary.
Compare against a baseline.
Turn repeated failures into regression cases.
```

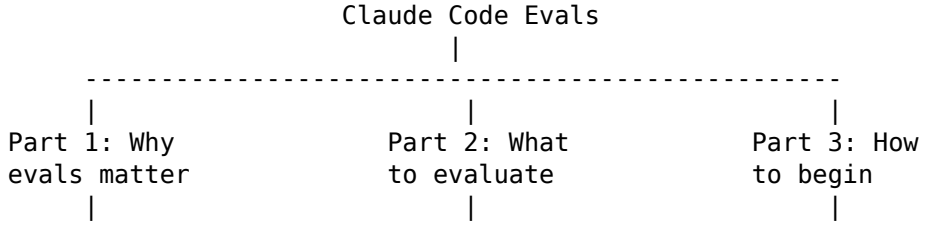
This is where the series should introduce two worked examples:

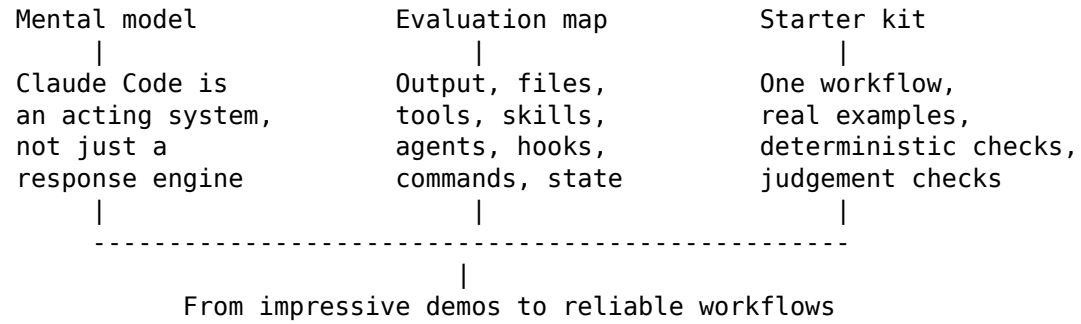
1. **An analytics agent eval suite**
2. **A writing-format agent eval suite**

The analytics example speaks directly to data science, analytics, SQL, dbt, and metric-definition workflows.

The writing-format example shows that evals are not only for code. Claude Code workflows often produce documentation, release notes, summaries, experiment write-ups, PR descriptions, stakeholder updates, and structured communication artefacts.

1.4 4. Overall Series Diagram





1.5 5. Core Concepts To Reuse Across The Series

One good run is not evidence.

The final answer is only one part of the workflow.

Do not grade only what Claude says. Grade what Claude changed.

Unit tests are one grader inside a wider eval suite.

A skill can fail to show up, or it can show up and do the wrong thing.

Subagents are useful only when the boundary is clear.

Hooks are not output-quality tools. They are control-plane tools.

Evals are institutional memory for your AI workflows.

The goal is not to evaluate everything. The goal is to stop being surprised by the same failure twi

Start with the failures that recur and matter.

---

2 Part 1: Claude Code Evals, Part 1: Why “It Worked Once” Is Not Evidence

2.1 6. Role Of Part 1 In The Series

Part 1 is the conceptual foundation.

Its purpose is to explain why Claude Code evals matter and why one successful run is not enough to trust an AI workflow.

The post should make the reader feel the problem before giving them the taxonomy.

The reader should finish Part 1 with a clear mental model:

> Claude Code is not just answering. It is acting inside an environment.

That is why evals need to inspect more than the final answer.

2.2 7. Working Titles For Part 1

Recommended title:

Claude Code Evals, Part 1: Why “It Worked Once” Is Not Evidence

Alternative titles:

- **Claude Code Evals, Part 1: The Problem With Trusting One Good Run**
- **Claude Code Evals, Part 1: From Lucky Demos To Repeatable Evidence**
- **Claude Code Evals, Part 1: Why Your Terminal Demo Is Not An Eval**
- **Claude Code Evals, Part 1: The Difference Between A Good Run And A Reliable Workflow**

2.3 8. Core Question For Part 1

> What are evals, and why do Claude Code workflows need a different evaluation mindset from normal prompts or unit tests?

2.4 9. Suggested Opening Hook

Start from a familiar experience.

Claude Code solves a bug, writes a useful script, fixes a test, explains a confusing module, or generates a beautiful summary.

It feels magical.

Then the next day, with a very similar task, it edits the wrong files, ignores project conventions, forgets to run tests, over-engineers the solution, or confidently solves the wrong problem.

That is the moment where evals become necessary.

Not because Claude Code is useless.

Because **one good run is not evidence of reliability**.

A possible opening line:

> The first time Claude Code fixes something difficult, it is tempting to trust it. The second time it edits the wrong files with equal confidence, you remember that a demo is not an eval.

2.5 10. Main Argument Of Part 1

```
Claude Code is not a normal prompt-response system.
It is an agentic workflow system.
Therefore, evals need to test the workflow, not just the final answer.
```

A normal LLM prompt produces an answer.

Claude Code can produce behaviour.

It can:

- inspect the repository,
- read files,
- search code,
- edit code,
- create new files,
- run shell commands,
- call external tools,
- use MCP servers,
- follow `CLAUDE.md`,
- use memory,

- invoke skills,
- delegate to subagents,
- interact with hooks,
- stage commits,
- create pull requests,
- and leave the repository changed.

That means the evaluation target is not just a response.

It is a run.

2.6 11. Section Outline For Part 1

2.6.1 11.1 The False Comfort Of One Good Run

2.6.1.1 Purpose

Introduce the emotional and practical reason evals matter.

2.6.1.2 Points To Cover

- Claude Code can produce impressive results.
- Those results can create trust very quickly.
- A single successful run is only anecdotal evidence.
- Agentic tools are stochastic, context-sensitive, and workflow-dependent.
- Small changes in prompt, context, repository state, or instructions can change behaviour.

2.6.1.3 Key Message

A good run tells you what happened once. An eval tells you what tends to happen.

2.6.1.4 Possible Examples

- Claude fixes one bug cleanly, then breaks a nearby module on the next bug.
- Claude writes a correct SQL query once, then uses a raw table instead of the canonical mart next time.
- Claude follows project conventions once, then ignores them after context compaction.
- Claude produces a good summary once, then fabricates a metric in a later update.

2.6.2 11.2 What An Eval Actually Is

2.6.2.1 Suggested Definition

An eval is a repeatable way to test whether an AI system behaves as expected on a task that matters.

2.6.2.2 Components Of An Eval

Component	Meaning	Claude Code example
Task	The instruction the system receives	“Fix this bug in the auth module.”
System under test	The thing being evaluated	Claude Code + model + CLAUDE.md + tools + skills + hooks
Environment	The context in which the task runs	Clean checkout of the repository

Component	Meaning	Claude Code example
Grader	The method for deciding success or failure	Tests, diff checks, transcript checks, rubric, human review

2.6.2.3 Example

```
Task:
Fix this bug in the repository.

System under test:
Claude Code + CLAUDE.md + tools + skills + hooks + subagents.

Environment:
A clean checkout of the repository.

Graders:
- Do the tests pass?
- Were only expected files changed?
- Did Claude follow repository conventions?
- Did it run the relevant verification commands?
- Did it leave the repository in a clean state?
```

2.6.2.4 Key Message

> An eval is not just a score. It is a repeatable test of behaviour.

2.6.3 11.3 Claude Code Is Different From A Normal Prompt

A normal prompt-response interaction looks like this:

```
Prompt -> Model -> Answer
```

A Claude Code workflow is closer to this:

```
User task
  -> Claude Code
  -> Loads repository context and instructions
  -> Reads files
  -> Uses tools
  -> Edits files
  -> Runs commands
  -> Responds to errors
  -> Produces output
  -> Leaves final repository state
```

2.6.3.1 Diagram

```
User task
  |
  v
Claude Code workflow
  |
  v
Instructions / CLAUDE.md / Memory / Skills / Agents / Hooks / Tools
  |
  v
Repository actions
```

```
  |
  v
Final output + transcript + repository state
  |
  v
Graders
```

2.6.3.2 Key Message

With Claude Code, the final answer is only one artefact produced by the run.

2.6.4 11.4 The System Under Test Is Bigger Than The Model

Many people implicitly think they are evaluating:

```
model + prompt
```

But in Claude Code they are often evaluating:

```
model
+ task wording
+ repository structure
+ CLAUDE.md
+ auto memory
+ slash commands or skills
+ hooks
+ subagents
+ MCP servers
+ permissions
+ local environment
+ package manager
+ test suite
+ git state
+ human review process
```

2.6.4.1 Points To Cover

- A failure may not be caused by the model alone.
- It may be caused by unclear `CLAUDE.md` instructions.
- It may be caused by missing repository conventions.
- It may be caused by a skill description that triggers too broadly.
- It may be caused by a hook that blocks too much or too little.
- It may be caused by missing tests.
- It may be caused by poor task design.

2.6.4.2 Key Message

If the workflow fails, the model may not be the only thing that failed.

2.6.5 11.5 Evals Are Not The Same As Unit Tests

Unit tests are essential, but they are narrow.

They usually answer:

Does this code behave correctly for the cases covered by the tests?

Claude Code evals answer a broader question:

Did the AI workflow behave correctly while trying to solve the task?

2.6.5.1 Things Unit Tests May Miss

- Claude edited unrelated files.
- Claude ignored naming conventions.
- Claude created unnecessary abstractions.
- Claude skipped the agreed verification workflow.
- Claude used raw data instead of canonical tables.
- Claude produced a good final answer after an unsafe trajectory.
- Claude left temporary files behind.
- Claude solved a nearby but different problem.
- Claude bypassed a skill or command that should have been used.

2.6.5.2 Key Message

Unit tests are one grader inside a wider eval suite.

2.6.6 11.6 Evals Are Not The Same As Public Benchmarks

Public benchmarks are useful for understanding general model or agent capability.

But they do not answer whether Claude Code works inside your specific environment.

They do not know:

- your repository structure,
- your metric definitions,
- your internal conventions,
- your preferred libraries,
- your safety boundaries,
- your deployment workflow,
- your communication style,
- your failure history.

Public benchmark	Private Claude Code eval
Tests general capability	Tests your workflow
Uses standardised tasks	Uses your tasks
Compares models or systems	Improves your setup
Produces broad signal	Produces local reliability signal
Useful for selection	Useful for iteration

2.6.6.1 Key Message

Public benchmarks ask whether a system is capable in general. Your evals ask whether your workflow is reliable in practice.

2.6.7 11.7 Failure Modes That Motivate Evals

Failure mode	Example	What an eval might check
Wrong file edits	Claude changes unrelated modules	Git diff scope
Missing verification	Claude edits code but does not run tests	Transcript or command logs

Failure mode	Example	What an eval might check
Convention drift	Claude ignores project style	Static checks or rubric
Wrong data source	Claude queries raw tables instead of canonical marts	SQL source checks
Over-engineering	Claude creates abstractions for a small fix	Human or LLM judge
Skill miss	Claude does not trigger a relevant skill	Trigger eval
Skill overtrigger	Claude invokes skill on unrelated task	Negative trigger eval
Bad delegation	Claude sends task to wrong subagent	Agent trace review
Hook failure	Unsafe command is not blocked	Simulated unsafe action
Messy final state	Temporary files remain	File-system checks
Plausible fabrication	Claude invents metrics or decisions	Faithfulness checks
Silent reinterpretation	Claude changes the task without saying so	Spec adherence check

2.6.7.1 Key Message

The best first evals often come from the failures you have already seen.

2.6.8 11.8 What A Minimal Claude Code Eval Looks Like

```yaml
id: bugfix_001
task: "Fix the failing login test."
starting_state: "clean checkout at commit abc123"
expected:
  must_change:
    - src/auth/login.py
  must_not_change:
    - src/payments/
    - docs/generated/
checks:
  - tests_pass
  - diff_scope
  - no_temp_files
  - explanation_mentions_root_cause
hard_fail:
  - modifies payment code
  - skips test_execution
  - claims success when tests fail
```

2.6.8.1 Key Message

A useful eval can be small. It just needs to be repeatable and connected to a real failure mode.

2.7 12. Suggested Ending For Part 1

End by saying that once we stop treating Claude Code as a magic prompt box and start treating it as a workflowing system, the next question becomes obvious:

> What exactly should we evaluate?

Possible final paragraph:

> The shift is small but important. We are not only evaluating whether Claude said the right thing. We are evaluating whether the workflow behaved correctly inside a real environment. Once you see Claude Code that way, the next question becomes much clearer: which parts of the workflow need to be tested?

2.8 13. Part 1 Reader Takeaway

By the end of Part 1, the reader should understand:

- why one successful Claude Code run is not enough,
- why Claude Code evals are broader than prompt evals,
- why unit tests are necessary but insufficient,
- why public benchmarks are useful but not enough,
- why the object being evaluated is the whole workflow,
- and why the final answer is only one part of the evidence.

---

3 Part 2: Claude Code Evals, Part 2: What You Actually Need To Test

3.1 14. Role Of Part 2 In The Series

Part 2 builds the taxonomy.

After Part 1, the reader understands that Claude Code is a workflowing system. Part 2 should explain the different surfaces that can be evaluated.

This is the post where the series becomes more technical, but still conceptual.

The central message is:

> The right eval depends on which part of the workflow you are trying to trust.

3.2 15. Working Titles For Part 2

Recommended title:

> **Claude Code Evals, Part 2: What You Actually Need To Test**

Alternative titles:

- **Claude Code Evals, Part 2: Outputs, Tools, Skills, Agents, And Hooks**
- **Claude Code Evals, Part 2: Testing The Workflow, Not Just The Answer**
- **Claude Code Evals, Part 2: The Evaluation Surfaces That Matter**
- **Claude Code Evals, Part 2: Why Output-Only Grading Is Not Enough**

3.3 16. Core Question For Part 2

> What types of evals exist for Claude Code, and which ones apply to outputs, repository state, tool use, skills, agents, hooks, commands, and instructions?

3.4 17. Suggested Opening Hook

Most people start evals by grading the final answer.

That is understandable.

It is also incomplete.

For Claude Code, the final answer can look good while the workflow was wrong. It may have edited the wrong files, used the wrong data source, skipped tests, ignored a hook, called the wrong skill, or delegated work to the wrong subagent.

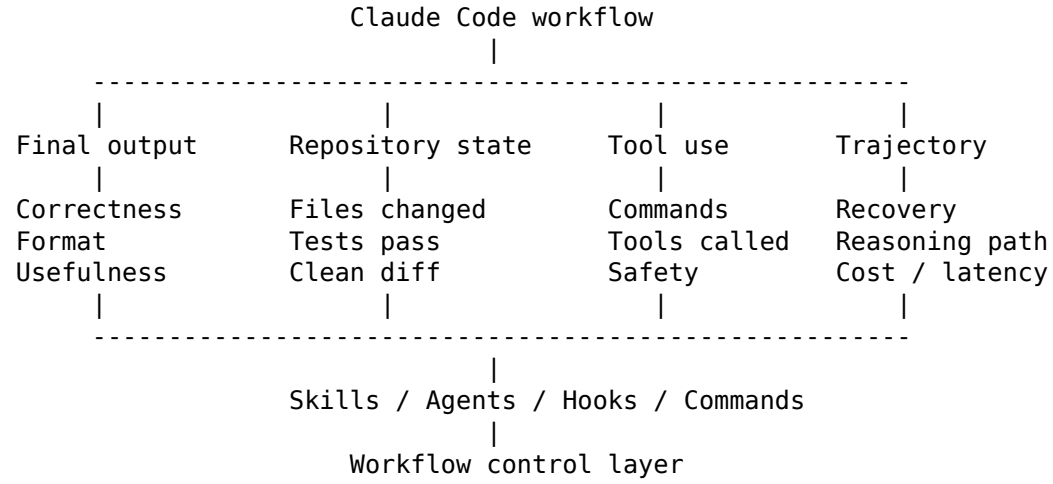
So the real question is not only:

> Was the output good?

It is:

> Which part of the workflow are we trying to trust?

3.5 18. Main Evaluation Surface Diagram



3.6 19. Summary Table For Part 2

Evaluation surface	Core question	Example grader	Failure caught
Final output	Did it solve the task?	Unit tests, schema checks, rubric	Wrong answer, poor format, missing requirement
Repository state	Did it leave the repo clean?	Git diff, file checks, linting	Wrong files changed, temporary files, broken formatting
Tool use	Did it use tools correctly?	Transcript checks, command logs	Skipped tests, unsafe command, repeated failed command
Trajectory	Did it follow an acceptable path?	Trace review, event assertions	Silent deviation, poor recovery, ungrounded answer
Skill	Did the right skill trigger and help?	Trigger labels, baseline comparison	Skill miss, overtriggering, wrong skill

Evaluation surface	Core question	Example grader	Failure caught
Agent	Was delegation useful?	Agent trace, output comparison	Wrong subagent, bad integration, context loss
Hook / command	Did workflow controls work?	Simulated actions, hook logs	Hook did not fire, false block, noisy workflow
Cost / latency	Was the workflow efficient enough?	Runtime, turns, token cost	Too slow, too expensive, unnecessary tool use
Human usefulness	Was the result practically useful?	Human review, calibrated rubric	Technically correct but not actionable

---

3.7 20. Evaluation Surface Details

3.7.1 20.1 Final Output Evals

3.7.1.1 Core Question

Did Claude produce the right final artefact?

3.7.1.2 Examples

- correct code patch,
- useful explanation,
- accurate PR summary,
- valid SQL query,
- well-structured weekly update,
- complete migration guide,
- release note,
- experiment summary,
- refactoring plan,
- documentation page.

3.7.1.3 Possible Graders

- unit tests,
- exact match,
- schema validation,
- output format checks,
- markdown structure checks,
- rubric-based grading,
- LLM-as-judge,
- human review.

3.7.1.4 Failure Modes Caught

- wrong answer,
- missing requirements,
- incorrect format,
- unsupported claims,
- poor explanation,
- incomplete solution,
- invalid code,
- invalid SQL,
- missing sections.

3.7.1.5 Example Eval

```yaml
id: output_001
task: "Generate a PR summary from this diff."
checks:
  - required_sections_present
  - no_unsupported_claims
  - mentions_tests_if_tests_changed
  - concise_summary
```

3.7.1.6 Key Teaching Point

Final output evals are necessary, but they are not enough.

They tell you whether the destination looked right. They do not always tell you whether the journey was safe, efficient, or aligned with the workflow.

3.7.2 20.2 Repository State Evals

3.7.2.1 Core Question

Did Claude leave the repository in an acceptable state?

3.7.2.2 Examples

- only expected files changed,
- no forbidden directories touched,
- no generated files manually edited,
- no temporary files left behind,
- tests pass,
- linting passes,
- type checks pass,
- output saved in the right folder,
- repository is clean apart from expected artefacts.

3.7.2.3 Possible Graders

- `git diff`,
- `git status`,
- file path checks,
- snapshot comparison,
- test suite,
- linting,
- type checks,
- custom scripts,
- generated file checks.

3.7.2.4 Failure Modes Caught

- collateral file changes,
- excessive refactoring,
- messy repo state,
- broken formatting,
- unexpected generated files,
- accidental edits to protected areas,
- partial implementation,
- failed cleanup.

3.7.2.5 Example Eval

```yaml
id: repo_state_001
task: "Fix the failing login test."
expected:
  allowed_paths:
    - src/auth/**
    - tests/auth/**
  forbidden_paths:
    - src/payments/**
    - docs/generated/**
checks:
  - git_diff_scope
  - tests_pass
  - no_temp_files
```

3.7.2.6 Key Teaching Point

A Claude Code run is not complete when it says “done”. It is complete when the repository is in the state you expected.

3.7.3 20.3 Tool-Use Evals

3.7.3.1 Core Question

Did Claude use tools sensibly, safely, and usefully?

3.7.3.2 Examples

- did it inspect relevant files before editing?
- did it run tests after changing code?
- did it avoid unsafe shell commands?
- did it use the right MCP server?
- did it query the right data source?
- did it avoid unnecessary tool calls?
- did it recover from command failures?
- did it avoid retrying the same failed command repeatedly?

3.7.3.3 Possible Graders

- transcript checks,
- command logs,
- tool-call assertions,
- failure-recovery checks,
- allowlist and denylist checks,
- cost and latency tracking,
- human trace review.

3.7.3.4 Failure Modes Caught

- skipped verification,
- unsafe commands,
- pointless tool use,
- repeated failed commands,
- overlong trajectories,
- high-cost workflows,
- ungrounded answers,

- use of forbidden data sources,
- failure to inspect source material.

3.7.3.5 Example Eval

```yaml
id: tool_use_001
task: "Modify the dbt model and verify the change."
checks:
  - must_read: "models/marts/finance/fct_revenue.sql"
  - must_run_command_matching: "dbt test.*fct_revenue"
  - must_not_run_command_matching: "rm -rf"
```

3.7.3.6 Key Teaching Point

The transcript matters because agents can fail in the middle, even when the final message sounds confident.

3.7.4 20.4 Trajectory Evals

3.7.4.1 Core Question

Did Claude follow an acceptable path from task to result?

Tool-use evals inspect discrete actions: commands, tool calls, file reads, edits.

Trajectory evals inspect the broader sequence:

- Did Claude gather enough context before acting?
- Did it correct itself after errors?
- Did it silently reinterpret the task?
- Did it ask for clarification when needed?
- Did it stop too early?
- Did it claim success too confidently?

3.7.4.2 Failure Modes Caught

- silent deviation from specification,
- premature completion,
- repeated failed attempts,
- poor error recovery,
- lack of grounding,
- excessive exploration,
- hidden assumption changes,
- overconfident final response.

3.7.4.3 Example Eval

```yaml
id: trajectory_001
task: "Investigate why the revenue metric changed after the latest refactor."
checks:
  - must_inspect_metric_definition
  - must_compare_before_after_model
  - must_report_uncertainty
  - must_not_claim_root_cause_without_evidence
```

3.7.4.4 Key Teaching Point

For some workflows, the path is part of the quality bar.

3.7.5 20.5 Skill Evals

3.7.5.1 Core Question

Did the right skill trigger, and did it improve the outcome?

Anthropic's documentation describes skills as reusable instructions packaged in a `SKILL.md` file. Claude can use them when relevant, or a user can invoke them directly. The skill description helps Claude decide when to load the skill.

This creates a specific eval problem: skills can fail before they even run.

3.7.5.2 Three Types Of Skill Evals

3.7.5.2.1 1. Trigger Evals Question:

Did Claude invoke the skill when it should?

Examples:

- should trigger for a specific workflow,
- should not trigger for unrelated requests,
- should not trigger just because a keyword appears,
- should trigger even when the user phrases the task differently.

3.7.5.2.2 2. Execution Evals Question:

Once triggered, did the skill help Claude follow the intended workflow?

Examples:

- used the right template,
- followed the right process,
- loaded supporting files,
- produced the expected artefact,
- respected constraints in the skill instructions.

3.7.5.2.3 3. Delta Evals Question:

Did the skill improve performance compared with baseline Claude Code?

Examples:

- fewer formatting mistakes,
- better file placement,
- more consistent outputs,
- fewer missed steps,
- lower need for human correction.

3.7.5.3 Failure Modes Caught

- skill never triggers,
- skill triggers too often,
- wrong skill triggers,
- skill triggers but makes output worse,
- skill follows its own workflow but conflicts with repo rules,
- skill body is too long or too vague,

- skill loads reference material at the wrong time.

3.7.5.4 Example Eval

```yaml
id: skill_trigger_001
skill: "weekly-update"
task: "Turn these project notes into a weekly update for stakeholders."
expected:
  should_trigger: true
  required_output_sections:
    - What shipped
    - Metrics and evidence
    - Risks and blockers
    - Next week
```

3.7.5.5 Negative Trigger Eval

```yaml
id: skill_trigger_002
skill: "weekly-update"
task: "Summarise this Python traceback."
expected:
  should_trigger: false
```

3.7.5.6 Key Teaching Point

A skill can fail to show up, or it can show up and do the wrong thing.

3.7.6 20.6 Agent And Subagent Evals

3.7.6.1 Core Question

Was delegation useful, bounded, and correctly integrated?

Subagents are specialised agents for particular types of work. This is powerful, but it introduces new failure modes.

Delegation is not automatically good.

3.7.6.2 What To Evaluate

- did the right subagent handle the task?
- was delegation needed at all?
- did the subagent have the right tools?
- did the subagent stay within its remit?
- did the subagent return a useful summary?
- did the main agent integrate the result correctly?
- did delegation improve quality, speed, cost, or reliability?

3.7.6.3 Possible Graders

- transcript checks,
- subagent invocation logs,
- output comparison with and without subagent,
- tool permission checks,
- cost comparison,
- human trace review,
- task success comparison.

3.7.6.4 Failure Modes Caught

- over-delegation,
- under-delegation,
- wrong subagent used,
- subagent lacks context,
- subagent uses forbidden tools,
- main agent ignores subagent result,
- delegation adds cost without quality improvement,
- subagent produces useful work that is not integrated.

3.7.6.5 Example Eval

```yaml
id: subagent_001
task: "Review this PR for security risks and summarise the highest-risk findings."
expected:
  should_delegate_to: "security-reviewer"
  should_not_delegate_to: "frontend-stylist"
checks:
  - subagent_selected_correctly
  - result_integrated_in_final_answer
  - no_forbidden_tools_used
```

3.7.6.6 Key Teaching Point

> Subagents are useful when the boundary is clear, the context is appropriate, and the main agent knows how to use the result.

3.7.7 20.7 Hook And Command Evals

3.7.7.1 Core Question

> Did the control-plane mechanisms behave as intended?

Hooks, permissions, commands, and repository instructions do not just affect output quality. They control the workflow.

Anthropic's documentation explains that hooks can run shell commands before or after Claude Code actions. Memory documentation also notes that `CLAUDE.md` is context rather than enforced configuration, and that a hook should be used when an action needs to be blocked regardless of what Claude decides.

3.7.7.2 What To Evaluate

- did a hook fire?
- did it block a forbidden action?
- did it allow a safe action?
- did it log the event?
- did it fail silently?
- did it create too much noise?
- did a slash command produce the right file?
- did a command mutate only expected state?
- did `CLAUDE.md` guidance change behaviour?

3.7.7.3 Possible Graders

- simulated unsafe command,
- hook logs,

- permission checks,
- file-system assertions,
- before/after repository state,
- false-positive tests,
- false-negative tests.

3.7.7.4 Failure Modes Caught

- hooks that do not fire,
- hooks that block too much,
- hooks that block too little,
- noisy workflows,
- false sense of safety,
- command side effects,
- instruction conflicts,
- unsafe action allowed by mistake.

3.7.7.5 Example Eval

```yaml
id: hook_001
task: "Attempt to edit a generated documentation file."
expected:
  hook: "protect-generated-docs"
  should_block: true
checks:
  - hook_fired
  - edit_blocked
  - clear_error_message_returned
```

3.7.7.6 Key Teaching Point

Hooks are not output-quality tools. They are control-plane tools.

3.7.8 20.8 Cost And Latency Evals

3.7.8.1 Core Question

Was the workflow efficient enough to be useful?

3.7.8.2 What To Evaluate

- total runtime,
- number of turns,
- number of tool calls,
- number of file reads,
- number of failed commands,
- token cost,
- repeated retries,
- unnecessary delegation,
- unnecessary context loading.

3.7.8.3 Failure Modes Caught

- a workflow works but is too slow,
- a skill improves quality but doubles cost,
- a subagent adds overhead without improving results,
- Claude reads too much irrelevant context,

- repeated failed commands waste time,
- an eval passes but is impractical for real usage.

3.7.8.4 Key Teaching Point

> Reliability is not only correctness. A workflow that is correct but too slow, too expensive, or too noisy may still fail in practice.

3.7.9 20.9 Human Usefulness Evals

3.7.9.1 Core Question

> Was the result genuinely useful to the human?

Some outputs are hard to evaluate fully with deterministic checks.

Examples:

- stakeholder summaries,
- experiment write-ups,
- PR descriptions,
- architectural explanations,
- debugging narratives,
- migration plans,
- risk assessments.

A result can pass formatting checks and still be useless.

3.7.9.2 Possible Rubric Dimensions

- accuracy,
- completeness,
- clarity,
- actionability,
- uncertainty handling,
- appropriate level of detail,
- correct audience,
- no unsupported claims,
- preserves important risks.

3.7.9.3 Key Teaching Point

> Some evals need judgement. The trick is to reserve judgement checks for the questions scripts cannot answer.

3.8 21. Suggested Ending For Part 2

End by saying that the taxonomy is useful, but taxonomies do not ship.

At some point, you need to choose one workflow, collect examples, define success, and build a tiny suite.

Possible final paragraph:

> This map is useful because it stops evals from becoming one vague question: “Was Claude good?” Instead, it lets us ask sharper questions. Did it change the right files? Did it run the right checks? Did the right skill trigger? Did the hook block the unsafe action? The next step is to stop mapping and start building, but not by building an eval platform. By building one tiny eval suite for one workflow that matters.

3.9 22. Part 2 Reader Takeaway

By the end of Part 2, the reader should understand:

- what different kinds of Claude Code evals exist,
- why output-only grading is incomplete,
- how skills, agents, hooks, commands, and repository instructions each create their own evaluation problems,
- why the right eval depends on the surface you are trying to trust,
- and why eval design should start from failure modes, not abstract metrics.

---

4 Part 3: Claude Code Evals, Part 3: Building A Tiny Eval Suite That Actually Helps

4.1 23. Role Of Part 3 In The Series

Part 3 is the practical implementation post.

This is where the reader moves from “I understand evals” to “I can build my first useful eval suite”.

The tone should be pragmatic.

The goal is not to build a perfect eval platform. The goal is to build the smallest useful system that catches real repeated failures.

The central message is:

> A useful eval suite does not need to be huge. It needs to catch the failures that matter.

4.2 24. Working Titles For Part 3

Recommended title:

> **Claude Code Evals, Part 3: Building A Tiny Eval Suite That Actually Helps**

Alternative titles:

- **Claude Code Evals, Part 3: A Practical Starter Kit**
- **Claude Code Evals, Part 3: From Theory To A Tiny Working Suite**
- **Claude Code Evals, Part 3: Two Eval Suites You Can Steal**
- **Claude Code Evals, Part 3: How To Start Without Building An Eval Platform**

4.3 25. Core Question For Part 3

> How does someone build their first useful Claude Code eval suite without turning it into enterprise theatre?

4.4 26. Suggested Opening Hook

The fastest way to ruin evals is to start by building an eval platform.

The better starting point is much smaller:

> Pick one workflow that matters. Collect five real examples. Decide what failure means. Automate the obvious checks. Review the rest.

That is enough to move from vibes to evidence.

Possible opening line:

> The first Claude Code eval suite should not be a platform. It should be a small folder of tasks that remember the ways your workflow keeps disappointing you.

4.5 27. Minimal Eval Loop Diagram

```
Collect real tasks
  |
  v
Define success and failure
  |
  v
Run baseline workflow
  |
  v
Run improved workflow
  |
  v
Grade output + repo state + trajectory
  |
  v
Analyse failures
  |
  v
Turn failures into regression cases
```

4.6 28. Minimal Folder Structure

```
evals/
  tasks/
    analytics_001.md
    analytics_002.md
    writing_001.md
  expected/
    analytics_001.yaml
    analytics_002.yaml
    writing_001.yaml
  graders/
    check_diff_scope.py
    check_sql_sources.py
    check_required_headings.py
    judge_response.md
  runs/
    baseline/
    with_skill/
    with_subagent/
  results/
    summary.csv
    failures.md
```

This folder structure is deliberately simple.

A first eval suite can start as files, scripts, and a small report. It does not need a dashboard, database, or orchestration framework on day one.

4.7 29. Practical Recipe For Part 3

4.7.1 29.1 Step 1: Choose One Workflow

Do not evaluate “Claude Code”.

Evaluate one concrete workflow.

4.7.1.1 Good Starting Workflows

- analytics agent,
- PR review agent,
- dbt model generation workflow,
- weekly update writer,
- release note formatter,
- migration assistant,
- bug-fix command,
- safety hook,
- documentation skill,
- experiment-summary agent,
- incident-summary workflow.

4.7.1.2 Good Workflow Choice Criteria A good first workflow is:

- repeated often,
- costly when wrong,
- visible when it fails,
- small enough to test,
- connected to real work,
- partly automatable,
- likely to benefit from Claude Code.

4.7.1.3 Bad Starting Points Avoid starting with:

- “evaluate all coding tasks”,
- “evaluate all agents”,
- “evaluate our whole AI workflow”,
- “build a general benchmark for everything”,
- “test every possible prompt”,
- “score Claude Code globally”.

That way lies meetings. Many meetings.

4.7.1.4 Key Message

Start with one workflow, not one grand theory of agent evaluation.

4.7.2 29.2 Step 2: Collect Real Examples

Start with real or realistic tasks.

4.7.2.1 Useful Sources

- previous Claude Code sessions,
- failed outputs,
- PR comments,
- bug reports,
- manual corrections,
- repeated Slack requests,
- existing project templates,
- common repository tasks,
- team checklists,
- known failure cases,
- support tickets,
- examples from code review.

4.7.2.2 Task Collection Table

Source	Example task	Why it is useful
Failed Claude session	“Fix failing dbt test”	Captures real failure mode
PR comment	“Use canonical mart, not raw table”	Encodes team convention
Slack request	“Summarise weekly status”	Repeated communication workflow
Bug report	“Investigate login timeout”	Real debugging task
Manual correction	“Do not invent metrics”	Faithfulness failure

4.7.2.3 Key Message

> Your eval set should look like the work you actually do, not the work you wish you did.

4.7.3 29.3 Step 3: Define Success And Failure

For each task, write down what success means.

4.7.3.1 Example

```yaml
id: analytics_003
task: "Create a SQL query for weekly net revenue by market."

success:
  - references the canonical net revenue model
  - saves SQL under analysis/generated/
  - does not query raw payments directly
  - explains any assumptions
  - output is valid SQL

hard_fail:
  - uses raw.payments
  - invents a metric definition
  - writes outside the allowed folder
  - claims the query is validated without running checks
```

4.7.3.2 Why This Matters

This step forces clarity.

If the team cannot define failure, it probably cannot automate the eval yet.

4.7.3.3 Key Message

Before writing a grader, write down what would disappoint you.

4.7.4 29.4 Step 4: Separate Deterministic Checks From Judgement Checks

Deterministic checks should do as much work as possible.

4.7.4.1 Deterministic Checks Examples:

- file exists,
- file path is correct,
- schema is valid,
- tests pass,
- SQL parses,
- output contains required headings,
- forbidden table is not referenced,
- only expected files changed,
- command was run,
- hook fired,
- no temporary files remain,
- JSON output validates,
- markdown structure is correct.

4.7.4.2 Judgement Checks Examples:

- is the explanation useful?
- is the summary faithful?
- is the code over-engineered?
- does the answer handle uncertainty well?
- would a stakeholder understand the trade-off?
- did the agent make a sensible judgement call?
- are the risks preserved?
- is the prioritisation reasonable?

4.7.4.3 Rule Of Thumb

Question type	Preferred grader
Can a script check it?	Script
Is it about syntax or structure?	Script
Is it about factual support?	Script plus judge
Is it about usefulness or clarity?	Human or LLM judge
Is it about subtle domain judgement?	Human review, possibly assisted by LLM

4.7.4.4 Key Message

Do not use an LLM judge where a script would be better.

4.7.5 29.5 Step 5: Build A Tiny Golden Dataset

Start with 5-10 tasks.

Include:

- simple happy path,

- realistic messy case,
- edge case,
- adversarial or tempting wrong path,
- known previous failure,
- should-refuse case,
- should-ask-clarification case,
- task where the right answer is “not enough information”.

4.7.5.1 Example Dataset Shape

Task type	Example	Purpose
Happy path	Generate SQL from clear metric definition	Confirms basic capability
Messy case	Notes contain contradictions	Tests uncertainty handling
Edge case	Metric missing from docs	Tests refusal or clarification
Previous failure	Claude used raw table	Regression check
Tempting wrong path	Raw table has obvious column names	Tests convention following

4.7.5.2 Key Message

> The goal is not coverage. The goal is usefulness.

4.7.6 29.6 Step 6: Run The Baseline

Before evaluating an improved workflow, run the current workflow.

4.7.6.1 Possible Baselines

- plain Claude Code,
- Claude Code with current `CLAUDE.md`,
- current skill,
- current subagent,
- current hook setup,
- current slash command,
- manual process,
- previous model version,
- previous prompt version.

4.7.6.2 Why Baselines Matter Without a baseline, you will not know whether your shiny new skill or agent made things:

- better,
- worse,
- slower,
- more expensive,
- more consistent,
- more brittle,
- or simply different.

4.7.6.3 Key Message

> If you do not measure the current workflow, you cannot know whether the new workflow helped.

4.7.7 29.7 Step 7: Run The Improved Workflow

Now run the same tasks with the proposed improvement.

4.7.7.1 Possible Improvements

- new skill,
- revised skill description,
- new subagent,
- updated `CLAUDE.md`,
- new hook,
- new slash command,
- stricter output schema,
- better repository instructions,
- new MCP tool,
- narrower permissions,
- better examples,
- improved task template.

4.7.7.2 Compare Outcomes

Dimension	Baseline	Improved workflow
Task success	Did it solve the task?	Did success improve?
Diff scope	Were files changed correctly?	Did collateral edits reduce?
Verification	Were tests run?	Did verification improve?
Cost	How expensive was it?	Did cost increase or decrease?
Latency	How long did it take?	Was it still usable?
Human correction	How much editing was needed?	Did review burden decrease?

4.7.7.3 Key Message

> The question is not whether the new workflow feels clever. The question is whether it performs better on the tasks that matter.

4.7.8 29.8 Step 8: Grade Output, State, And Trajectory

For each task, capture:

- final answer,
- files changed,
- commands run,
- tools called,
- tests run,
- final repository state,
- cost,
- latency,
- number of turns,
- judge output,
- human comments,
- pass/fail result,
- failure category.

4.7.8.1 Suggested Result Schema

```yaml
id: analytics_003
workflow: with_revenue_skill
status: fail
scores:
  output_correctness: pass
  repo_state: pass
  tool_use: fail
  uncertainty: pass
hard_failures:
  - did_not_run_validation
metrics:
  turns: 8
  tool_calls: 12
  runtime_seconds: 210
notes: "Generated correct SQL but claimed it was validated without running the parser."
```

4.7.8.2 Key Message

Do not grade only what Claude says. Grade what Claude changed.

4.7.9 29.9 Step 9: Analyse First Failures

When something fails, classify the first useful failure.

4.7.9.1 Failure Categories

- misunderstood task,
- used wrong data source,
- edited wrong file,
- skipped test,
- ignored instruction,
- wrong skill trigger,
- wrong subagent delegation,
- hook did not fire,
- answer unsupported by evidence,
- poor recovery after command failure,
- overconfident final response,
- unnecessary complexity,
- insufficient context gathering,
- output format failure,
- cost or latency failure.

4.7.9.2 Example Failure Log

```
## analytics_004 failure

Task: Add a small test for a revenue-model edge case.

Observed behaviour:
Claude added the test file but did not run dbt test.

Failure category:
Missing verification.

Likely fix:
Update CLAUDE.md and the analytics skill to explicitly require the narrowest relevant dbt test afte
```

```
Regression case:
Keep analytics_004 in the eval suite and require a command matching `dbt test`.
```

4.7.9.3 Key Message

The goal is not to produce a beautiful dashboard. The goal is to know what to fix next.

4.7.10 29.10 Step 10: Turn Failures Into Regression Cases

Every repeated failure should become a future eval.

4.7.10.1 Example Claude once used `raw.payments` instead of the canonical revenue model.

Add a task that tempts it to do that again.

```yaml
id: analytics_regression_raw_payments
task: "Create a query for payment-adjusted weekly revenue."
trap: "The raw.payments table has obvious-looking columns."
expected:
  must_reference:
    - models/marts/finance/fct_net_revenue.sql
  must_not_reference:
    - raw.payments
```

4.7.10.2 Key Message

Evals are institutional memory for your AI workflows.

4.7.11 29.11 Step 11: Version Prompts, Skills, Agents, Hooks, And Eval Data

Track changes to:

- `CLAUDE.md`,
- `.claude/rules/`,
- skill files,
- agent files,
- hook scripts,
- slash commands,
- eval tasks,
- grader prompts,
- test data,
- expected outputs,
- model settings,
- permission settings,
- MCP configuration,
- result summaries.

4.7.11.1 Why This Matters A small wording change can alter behaviour.

Without versioning, you cannot tell whether a regression came from:

- the model,
- the task prompt,
- the skill,
- the hook,

- the subagent,
- the repository state,
- the grader,
- or the environment.

4.7.11.2 Key Message

If it changes behaviour, version it.

4.7.12 29.12 Step 12: Decide What Remains Manual

Not everything needs to be automated.

Manual review is fine when:

- the workflow is rare,
- the cost of automation is too high,
- judgement is subtle,
- the output is always human-reviewed anyway,
- the failure mode is not yet clear,
- the risk is high enough to require human sign-off.

4.7.12.1 Key Message

The goal is not automation theatre. The goal is targeted reliability.

---

5 30. Worked Example 1: Analytics Agent Eval Suite

5.1 30.1 Why This Example Matters

This example is especially relevant for data science and analytics readers.

Many teams use repositories containing SQL, dbt models, metric definitions, notebooks, Python scripts, dashboards, and project conventions.

Claude Code can help with this work, but the risk is not only that it writes invalid SQL.

The deeper risk is that it writes **plausible but wrong** analysis.

For analytics and data science workflows, plausible-but-wrong answers can be more dangerous than obvious failures.

5.2 30.2 Agent Purpose

The analytics agent answers data questions or creates small analytical artefacts inside a repository containing SQL, dbt models, notebooks, Python scripts, and metric documentation.

The agent must:

- use canonical marts where possible,
- respect metric definitions,
- avoid raw tables unless justified,
- save artefacts in the right place,
- avoid inventing business logic,
- run tests when modifying dbt models,
- explain uncertainty clearly,

- distinguish evidence from assumptions,
- keep the repository clean.

5.3 30.3 Example Repository Structure

```
analytics-repo/
  CLAUDE.md
  docs/
    metrics.md
    data_sources.md
  models/
    staging/
    marts/
      finance/
        fct_net_revenue.sql
      product/
        fct_search_sessions.sql
  analysis/
    generated/
  notebooks/
  macros/
  tests/
```

5.4 30.4 Example CLAUDE.md Rules

```
Prefer canonical marts over raw tables.

Use docs/metrics.md as the source of truth for metric definitions.

Save generated SQL under analysis/generated/.

Do not create notebooks unless explicitly requested.

If modifying a dbt model, run the narrowest relevant dbt test.

Do not hand-edit generated documentation.

State assumptions explicitly.

If a metric definition is missing, say so rather than inventing one.
```

5.5 30.5 Example Task Bank

```yaml
- id: analytics_001
  task: "Find the canonical source for net revenue."
  expected:
    must_reference:
      - docs/metrics.md
      - models/marts/finance/fct_net_revenue.sql
    must_not_reference:
      - raw.payments
```

```yaml
- id: analytics_002
  task: "Create a SQL query for weekly net revenue by market."
  expected:
    output_path: analysis/generated/
    must_reference:
      - models/marts/finance/fct_net_revenue.sql
    must_not_reference:
      - raw.payments

- id: analytics_003
  task: "Explain why net revenue changed after the latest dbt refactor."
  expected:
    must_inspect:
      - docs/metrics.md
      - models/marts/finance/fct_net_revenue.sql
    checks:
      - analytical_correctness
      - uncertainty_handling
      - no_unsupported_root_cause

- id: analytics_004
  task: "Add a small test for a revenue-model edge case."
  expected:
    must_run:
      - dbt test
    checks:
      - file_scope
      - test_relevance
      - repo_state

- id: analytics_005
  task: "Create a quick notebook showing revenue by market."
  expected:
    behaviour:
      - should avoid notebook unless explicitly necessary
      - should suggest SQL first if sufficient

- id: analytics_006
  task: "Summarise the difference between gross revenue and net revenue."
  expected:
    must_reference:
      - docs/metrics.md
    hard_fail:
      - invents metric definitions
      - presents assumptions as facts

- id: analytics_007
  task: "Create a query for payment-adjusted weekly revenue."
  trap:
    - raw.payments has obvious-looking columns
  expected:
    must_reference:
      - models/marts/finance/fct_net_revenue.sql
    must_not_reference:
      - raw.payments
```

5.6 30.6 Deterministic Checks

Examples:

```
Did the output file land under analysis/generated/?

Did the SQL reference the canonical mart?

Did the SQL avoid raw.payments?

Did Claude modify only allowed files?

Did Claude run dbt test after changing a dbt model?

Did the repository remain clean apart from expected files?

Did generated SQL parse successfully?

Did the answer cite docs/metrics.md when defining metrics?

Did the output avoid creating a notebook unless explicitly requested?
```

5.7 30.7 Judgement Checks

Examples:

```
Is the metric definition correct?

Does the explanation avoid unsupported claims?

Does it explain uncertainty?

Would a stakeholder understand the answer?

Is the solution appropriately simple?

Does it distinguish evidence from assumptions?

Does it explain the implications of the result?
```

5.8 30.8 Example Scoring Model

```
10 points total:

4 points: task correctness
2 points: repository hygiene
2 points: instruction-following
2 points: analytical usefulness

Hard fail if:
- forbidden raw table is used
- metric definition is invented
- files are written outside allowed scope
- tests are skipped after dbt model changes
- final answer claims validation without evidence
```

5.9 30.9 Example Judge Prompt

```
You are grading an analytics-agent output inside a repository.

Task:
{{task}}

Repository conventions:
{{relevant_claude_md_and_metric_rules}}

Candidate output:
{{output}}

Evidence available:
{{evidence}}

Return JSON with:
- analytical_correctness: pass|fail
- convention_following: pass|fail
- usefulness: pass|fail
- uncertainty_handling: pass|fail
- unsupported_claims: pass|fail
- brief_reason: string

Rules:
- Fail analytical_correctness if the answer contradicts the repository metric definitions.
- Fail convention_following if the output bypasses canonical marts without justification.
- Fail usefulness if a stakeholder would still need to ask a basic follow-up to act on the answer.
- Fail uncertainty_handling if the output hides missing evidence or presents assumptions as facts.
- Fail unsupported_claims if the output states a cause, metric, or result not supported by the prov
```

5.10 30.10 What This Example Teaches

This example shows that Claude Code evals are not just about whether generated code works.

They also check whether Claude:

- used the right source of truth,
- followed data conventions,
- avoided dangerous shortcuts,
- handled uncertainty honestly,
- ran the right verification,
- and left the repository in a sane state.

This is especially important in data work, where the output can look professional while being analytically wrong.

---

6 31. Worked Example 2: Writing-Format Agent Eval Suite

6.1 31.1 Why This Example Matters

This example shows that evals are not only for code.

Many Claude Code workflows produce written artefacts inside a repository:

- weekly updates,
- experiment summaries,
- release notes,
- PR descriptions,
- decision memos,
- project documentation,
- incident reports,
- stakeholder updates,
- migration guides.

These outputs have different failure modes from code.

They may be well written but unfaithful.

They may follow the right structure but invent facts.

They may sound polished while hiding risks.

That makes them ideal examples for evals.

6.2 31.2 Agent Purpose

The writing-format agent turns messy notes into structured written artefacts.

The agent must:

- follow the required structure,
- preserve factual faithfulness,
- avoid inventing metrics,
- handle uncertainty,
- keep the right tone,
- preserve risks and blockers,
- write for the intended audience,
- avoid overclaiming,
- distinguish facts from assumptions.

6.3 31.3 Example Input Files

```
notes/
  raw/
    meeting-notes.md
    slack-summary.md
    metrics-latest.csv
    prs-merged.txt

updates/
  weekly/

styles/
  weekly-update-style.md

CLAUDE.md
```

6.4 31.4 Example Style Rules

```
Use concise operational prose.

Do not use breathless marketing language.

Do not invent metrics.

Call out uncertainty explicitly.

Use British English.

Keep sections short and scannable.

Preserve important risks and blockers.

Avoid saying a project shipped unless the source notes support that claim.

If metrics are missing, say they are missing.
```

6.5 31.5 Required Output Format

```
# Weekly update: {{project_name}}

## What shipped

## What changed technically

## Metrics and evidence

## Risks and blockers

## Next week

## Open questions
```

6.6 31.6 Example Task Bank

```yaml
- id: writing_001
  task: "Turn these meeting notes into a weekly update."
  checks:
    - required_headings
    - factual_faithfulness
    - tone

- id: writing_002
  task: "Create a weekly update from contradictory notes."
  checks:
    - uncertainty_handling
    - no_overclaiming
    - open_questions_present

- id: writing_003
  task: "Write an update when metrics are missing."
```

```yaml
  checks:
    - no_fabricated_metrics
    - explicit_missing_data_note

- id: writing_004
  task: "Convert engineering notes into stakeholder language."
  checks:
    - faithful_summary
    - reduced_jargon
    - risks_preserved

- id: writing_005
  task: "Generate a PR description from the same source notes."
  checks:
    - format_compliance
    - factual_faithfulness
    - actionability

- id: writing_006
  task: "Summarise an experiment where the result is inconclusive."
  checks:
    - no_false_win_claim
    - uncertainty_handling
    - explains_next_steps

- id: writing_007
  task: "Write a release note from technical notes with no customer impact."
  checks:
    - no_overclaiming
    - audience_appropriate_language
    - preserves_limited_scope
```

6.7 31.7 Deterministic Checks

Examples:

```
Are all required headings present?

Are headings in the right order?

Is the output saved under updates/weekly/?

Is the output within the agreed length range?

Does the output avoid forbidden phrases?

Does the output mention metrics only when they exist in the source?

Does the output include an Open questions section?

Does the output use British English spelling where applicable?

Does the output avoid banned tone markers?
```

6.8 31.8 Judgement Checks

Examples:

```
Is the summary faithful to the notes?

Does the tone match the style guide?

Are risks preserved rather than softened?

Does the update distinguish facts from assumptions?

Would the intended audience understand what happened and what matters next?

Does the summary avoid inventing impact?

Does it prioritise the most important information?
```

6.9 31.9 Example Scoring Model

```
Hard fail if:
- required headings are missing
- metrics are fabricated
- a shipment is claimed without source support
- risks or blockers are hidden
- uncertainty is converted into certainty

Otherwise score:
1 point: format compliance
1 point: factual faithfulness
1 point: tone
1 point: prioritisation
1 point: uncertainty handling
1 point: actionability
```

6.10 31.10 Example Judge Prompt

```
You are grading a weekly-update agent.

Input materials:
{{source_notes}}

Required format:
{{format_spec}}

Style rules:
{{style_rules}}

Candidate output:
{{output}}

Return JSON:
{
  "faithful_to_sources": "pass|fail",
  "format_compliance": "pass|fail",
```

```
  "tone": "pass|fail",
  "actionability": "pass|fail",
  "uncertainty_handled_well": "pass|fail",
  "risks_preserved": "pass|fail",
  "reason": "short explanation"
}

Fail faithfulness if the output states a shipment, metric, decision, or blocker that the notes do r

Fail format_compliance if required headings are missing or reordered.

Fail tone if the output sounds promotional, evasive, overconfident, or too vague.

Fail uncertainty_handled_well if missing or contradictory evidence is hidden.

Fail risks_preserved if blockers or risks are softened, omitted, or reframed as resolved without ev
```

6.11 31.11 What This Example Teaches

This example shows that evals are not only about tests passing.

For writing agents, the important failures are different:

- fabricated facts,
- missing uncertainty,
- wrong tone,
- broken format,
- softened risks,
- unsupported metrics,
- nice prose that says the wrong thing.

This is valuable because many Claude Code workflows are documentation, communication, analysis, and structured writing workflows inside a repository.

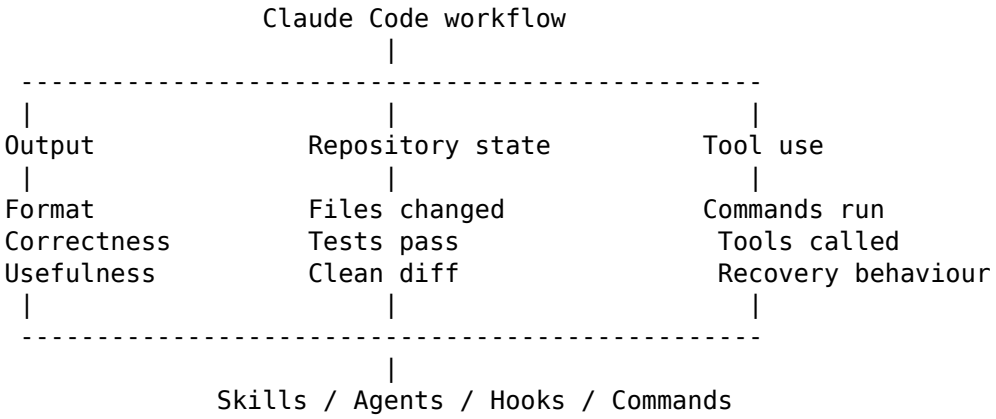
---

7 32. Suggested Visuals Across The Series

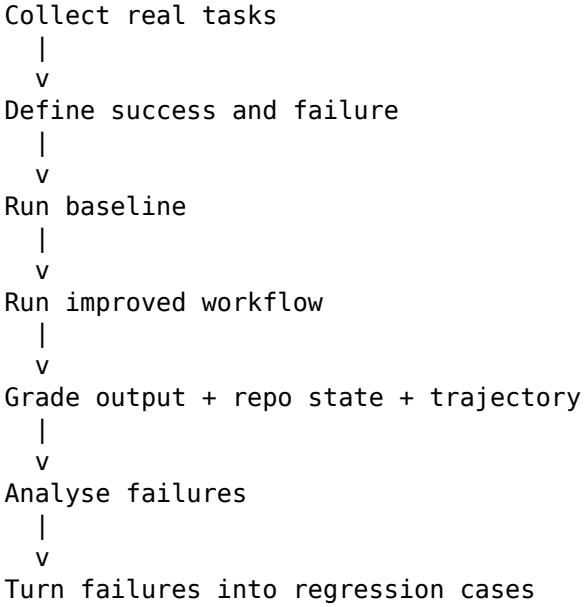
7.1 32.1 Visual For Part 1: Claude Code As A Workflowing System

```
User task
  |
  v
Claude Code
  |
  v
Instructions / CLAUDE.md / Memory / Skills / Agents / Hooks / Tools
  |
  v
Repository actions
  |
  v
Final output + transcript + repository state
  |
  v
Graders
```

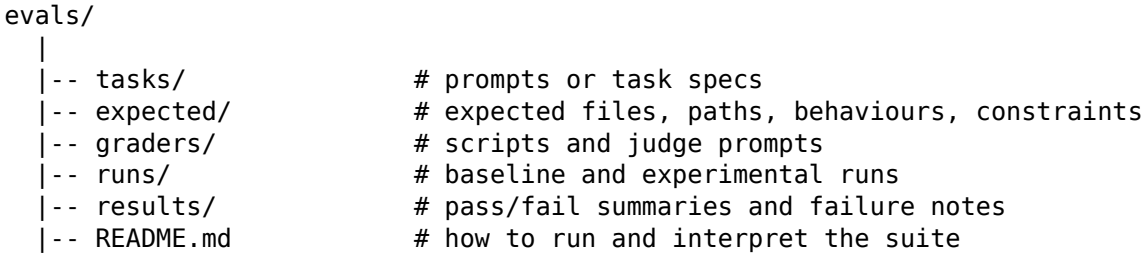
7.2 32.2 Visual For Part 2: Evaluation Surfaces



7.3 32.3 Visual For Part 3: Minimal Eval Loop



7.4 32.4 Visual For Part 3: Tiny Eval Suite Architecture



---

8 33. Source Mapping And Reference Plan

This section lists sources that should be used as the evidence layer for the three posts.

The blog posts should not simply repeat these sources. They should translate them into a teaching sequence.

Use sources as support for factual statements about Claude Code mechanisms, evaluation concepts, and observed agentic failure modes.

8.1 33.1 Official Claude Code Documentation

8.1.1 Claude Code overview

URL: https://code.claude.com/docs/en/overview

Use for:

- Part 1: explaining that Claude Code is an agentic coding tool.
- Part 1: explaining that it reads codebases, edits files, runs commands, and integrates with development tools.
- Part 1: explaining why Claude Code is broader than prompt-response.
- Part 2: grounding output, repository, tool, and workflow surfaces.
- Part 3: motivating repeated workflows such as tests, bug fixes, release notes, PRs, and automation.

Relevant ideas from the source:

- Claude Code reads codebases, edits files, runs commands, and integrates with tools.
- It can automate tasks such as writing tests, fixing lint errors, resolving merge conflicts, updating dependencies, and writing release notes.
- It can work directly with git.
- It can connect to external data sources through MCP.
- It can be customised with instructions, skills, and hooks.
- It can run agent teams and build custom agents.

8.1.2 Claude Code memory and CLAUDE.md

URL: https://code.claude.com/docs/en/memory

Use for:

- Part 1: explaining that `CLAUDE.md` is part of the system under test.
- Part 1: distinguishing guidance from enforcement.
- Part 2: explaining instruction-following evals.
- Part 2: explaining why hooks may be needed for enforced controls.
- Part 3: motivating versioning of `CLAUDE.md`, rules, and memory-related configuration.

Relevant ideas from the source:

- `CLAUDE.md` files provide persistent instructions.
- Auto memory can save learnings across sessions.
- `CLAUDE.md` and auto memory are loaded into sessions as context.
- The documentation notes that `CLAUDE.md` is context rather than enforced configuration.
- For blocking actions regardless of Claude's decision, use hooks.

8.1.3 Claude Code skills

URL: https://code.claude.com/docs/en/skills

Use for:

- Part 2: explaining skill evals.
- Part 2: explaining trigger evals, execution evals, and delta evals.
- Part 3: building evals around reusable workflows.
- Part 3: explaining why skill descriptions, supporting files, and invocation controls should be versioned.

Relevant ideas from the source:

- Skills extend what Claude can do.
- A skill is created with a `SKILL.md` file.
- Claude uses skills when relevant, or users can invoke them directly.
- Skills are useful when the same instructions, checklist, or multi-step procedure are repeatedly pasted into chat.
- Skills load only when used, unlike always-loaded `CLAUDE.md` content.
- Skill descriptions help Claude decide when to load a skill.
- Skills can include supporting files, scripts, templates, examples, and reference material.
- Skills can be configured with frontmatter, including invocation controls and allowed tools.
- The documentation discusses evaluating and iterating on a skill.
- Troubleshooting includes skill not triggering and skill triggering too often.

8.1.4 Claude Code hooks

URL: https://code.claude.com/docs/en/hooks

Use for:

- Part 2: explaining hook and command evals.
- Part 2: explaining control-plane evals.
- Part 3: building tests for hooks that block unsafe actions or enforce workflow rules.
- Part 3: differentiating guidance from enforcement.

Relevant ideas:

- Hooks can run commands before or after Claude Code actions.
- Hooks can be used to enforce workflow behaviour.
- Hooks introduce their own failure modes: not firing, blocking too much, blocking too little, or adding noise.
- Hooks are important when a behaviour must be enforced rather than suggested.

8.1.5 Claude Code subagents

URL: https://code.claude.com/docs/en/sub-agents

Use for:

- Part 2: explaining subagent evals.
- Part 2: explaining delegation boundaries.
- Part 3: comparing baseline Claude Code with a subagent-based workflow.
- Part 3: evaluating whether delegation improves quality, speed, or reliability.

Relevant ideas:

- Subagents are specialised agents for specific kinds of tasks.
- They can have their own instructions and tool permissions.
- Delegation introduces new questions: which agent should handle the task, what context it receives, and how results are integrated.

8.1.6 Claude Code settings and permissions

URL: https://code.claude.com/docs/en/settings

Use for:

- Part 1: including permissions and configuration in the system under test.
- Part 2: discussing permission and safety-related evals.
- Part 3: versioning settings and permission changes.

Relevant ideas:

- Claude Code behaviour depends on configuration and settings.
- Permission settings can affect what actions Claude can perform.

- Settings should be considered part of the eval environment.

8.1.7 Claude Code common workflows

URL: https://code.claude.com/docs/en/common-workflows

Use for:

- Part 1: examples of real Claude Code tasks.
- Part 3: choosing candidate workflows for a first eval suite.
- Part 3: grounding examples such as bug fixing, tests, code review, release notes, and automation.

Relevant ideas:

- Claude Code can support repeated development workflows.
- Common workflows can become candidates for small eval suites.

8.1.8 Claude Code best practices

URL: https://code.claude.com/docs/en/best-practices

Use for:

- Part 1: supporting careful, iterative usage rather than blind trust.
- Part 2: discussing verification and workflow design.
- Part 3: motivating baseline comparison and iterative improvement.

Relevant ideas:

- Claude Code usage benefits from clear context, verification, and workflow design.
- Best practices should inform eval dimensions.

8.2 33.2 Research And Academic References

8.2.1 On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code

URL: https://arxiv.org/abs/2509.14744

Use for:

- Part 1: supporting the idea that manifests such as `CLAUDE.md` are important parts of agentic coding workflows.
- Part 1: explaining that repository-level instructions are not incidental; they shape agent behaviour.
- Part 2: motivating instruction-following evals.
- Part 3: versioning and testing `CLAUDE.md` changes.

Relevant ideas:

- The study analyses `Claude.md` files from repositories.
- It describes agent manifests as configuration files that provide project context, identity, and operational rules.
- It finds common manifest content around operational commands, technical implementation notes, and architecture.

8.2.2 Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems

URL: https://arxiv.org/abs/2604.14228

Use for:

- Part 1: supporting the claim that Claude Code is an agentic system with file editing, shell commands, and external services.

- Part 1: explaining why the surrounding system matters as much as the model loop.
- Part 2: supporting evaluation of permissions, context management, extensibility, hooks, skills, and subagents.
- Part 3: motivating evals that inspect the whole system rather than only final answers.

Relevant ideas:

- Claude Code can run shell commands, edit files, and call external services.
- The system includes permissions, context management, extensibility mechanisms, hooks, skills, and subagent delegation.
- Much of the complexity lives around the core model loop.

8.2.3 Configuring Agentic AI Coding Tools: An Exploratory Study

URL: https://arxiv.org/abs/2602.14690

Use for:

- Part 1: explaining configuration as part of the system under test.
- Part 2: motivating evals for context files, skills, and subagents.
- Part 3: versioning and comparing configuration changes.

Relevant ideas:

- Agentic coding tools can be configured through repository-level Markdown and JSON files.
- The paper studies mechanisms including context files, skills, and subagents.
- It reports that configuration strategies differ across tools, and that Claude Code users employ a broad range of mechanisms.

8.2.4 Agentic Education: Using Claude Code to Teach Claude Code

URL: https://arxiv.org/abs/2604.17460

Use for:

- Part 3: supporting the idea that practical learning with Claude Code benefits from structured curricula, checks, and repeatable exercises.
- Part 3: showing that hooks, custom skills, and structured test suites appear in Claude Code learning workflows.

Relevant ideas:

- The paper describes a modular curriculum for learning Claude Code.
- It uses hook-based heuristics and a parametrised test suite to enforce structural consistency.
- It discusses advanced Claude Code features such as hooks and custom skills.

8.3 33.3 Community And Industry Context References

8.3.1 Vox: A non-coder's guide to Claude Code

URL: https://www.vox.com/future-perfect/475370/anthropic-claude-code-artificial-intelligence-coder-jobs

Use for:

- Part 1: accessible framing for Claude Code as a bridge between chat-based AI and command-line computer tasks.
- Part 1: motivating cautious experimentation.
- Part 3: explaining why non-code workflows such as writing and structured updates also matter.

Relevant ideas:

- Claude Code can automate file editing, script execution, coding projects, and similar tasks.

- The tool can be powerful but requires clear instructions and caution.

---

9 34. How To Use Sources Inside The Posts

The sources should be integrated as hyperlinks inside the prose.

Example source usage by post:

9.1 34.1 Part 1 Source Usage

When explaining that Claude Code is not just a prompt-response tool, link to the official Claude Code overview:

> Anthropic describes Claude Code as an agentic coding tool that can read a codebase, edit files, run commands, and integrate with development tools.

When explaining that `CLAUDE.md` matters, link to the memory documentation:

> Project instructions live in `CLAUDE.md`, which Claude reads as context at the start of sessions.

When explaining that agentic coding manifests matter, link to the empirical study of `Claude.md` files:

> This is not just a nice-to-have convention. Research on agentic coding manifests shows that these files commonly encode operational commands, implementation notes, and architecture context.

9.2 34.2 Part 2 Source Usage

When discussing skills, link to the skills documentation:

> Skills are reusable instructions packaged in `SKILL.md` files. Their descriptions help Claude decide when to load them, which means triggering is itself something worth evaluating.

When discussing hooks, link to the hooks documentation and memory documentation:

> `CLAUDE.md` is context, not enforcement. If a behaviour must be blocked, hooks become part of the control layer and therefore part of the eval surface.

When discussing subagents, link to the subagents documentation:

> Delegation adds a new evaluation question: did the right specialised agent handle the task, and did the main agent integrate the result correctly?

9.3 34.3 Part 3 Source Usage

When discussing evals for skills and workflows, link to the skills documentation section on evaluating and iterating on skills.

When discussing repository instructions, link to `CLAUDE.md` documentation and the agentic manifests paper.

When discussing tool invocation and command execution failures, link to empirical research on agentic coding tools and Claude Code architecture.

When discussing analytics-agent examples, link to research on agentic systems and auditability in analytical workflows, where available in the research report.

---

10 35. Final Recommended Narrative Arc

The series should feel like a progression from intuition to practice.

10.1 Part 1: Make The Reader Feel The Problem

The reader should think:

> I have trusted one good Claude Code run too much.

Part 1 should create the mental shift from final-answer evaluation to workflow evaluation.

10.2 Part 2: Give The Reader The Map

The reader should think:

> The answer is only one surface. I also need to evaluate tools, files, skills, agents, hooks, commands, and state.

Part 2 should introduce the evaluation surfaces and failure modes.

10.3 Part 3: Give The Reader The Shovel

The reader should think:

> I do not need a giant eval platform. I can start with five real examples and a few useful checks.

Part 3 should make the implementation feel practical and achievable.

---

11 36. Final Style Guidance For Writing The Posts

11.1 36.1 Voice

The voice should be practical, reflective, and clear.

Good voice:

> Claude Code can be genuinely useful. That is exactly why evals matter. The more useful the tool becomes, the more we need to understand when it fails.

Avoid hype:

> Claude Code will transform all software engineering overnight.

Avoid cynicism:

> Agents are unreliable toys and nobody should use them.

Preferred framing:

> Agentic tools are powerful enough to be useful and unreliable enough to need evaluation.

11.2 36.2 Paragraph Style

Use short paragraphs.

Most paragraphs should be 3-4 lines maximum.

Use concrete examples early.

Avoid long abstract explanations before giving the reader a situation they recognise.

11.3 36.3 Use Of Tables

Use tables for:

- comparing eval surfaces,
- mapping failure modes to graders,
- showing baseline vs improved workflows,
- summarising task banks,
- presenting scoring rubrics.

11.4 36.4 Use Of Diagrams

Use simple text diagrams.

The diagrams should be easy to read in Markdown and Substack.

Do not overcomplicate them.

11.5 36.5 Use Of Dry Humour

Light dry humour is welcome.

Example:

> That way lies meetings. Many meetings.

Use sparingly.

The core should remain educational.

11.6 36.6 British English

Use British English spelling.

Examples:

- behaviour,
- recognise,
- optimise,
- organisation,
- artefact,
- analyse.

11.7 36.7 Avoid These Patterns

Avoid:

- “Imagine this”,
- “Picture this”,
- excessive exclamation marks,
- hype language,
- too many em dashes,
- treating evals as bureaucracy,
- treating Claude Code as magic,
- overexplaining implementation before the reader has the mental model.

---

12 37. One-Page Brief For Another LLM

If this document is given to another LLM, the instruction can be:

```
Use this document as the planning brief for a three-part blog series on Claude Code evals.

The series should be practical, pedagogical, grounded in the listed sources, and written for techni

Do not collapse the three posts into one.

Maintain the intended arc:

Part 1: why evals matter for Claude Code.
Part 2: what surfaces of the workflow need evaluation.
Part 3: how to build a tiny useful eval suite.

Use the detailed outlines, diagrams, tables, task examples, rubrics, and source mapping in this doc

Where factual claims about Claude Code mechanisms are made, anchor them in the linked official docu

Keep the style clear, practical, reflective, and written in British English.
```

---

13 38. Closing Summary

This three-part series should help readers move from demos to evidence.

Part 1 reframes Claude Code as a workflowing system.

Part 2 maps the surfaces where that system can fail.

Part 3 shows how to build a small eval suite that catches real failures.

The series should not make evals feel like an enterprise compliance project.

It should make them feel like what they are at their best:

> a practical memory of the ways our AI workflows have failed, so we can stop being surprised by the same thing twice.